

开发工具链与 Git 工作流

DevOps · 版本控制 · 从代码到线上

一次推送，8 小时，850 万台电脑蓝屏，全球航班停飞——只因为缺少一个测试环节.....

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

模块	关键技术
DevOps 开发文化	理念 · 工具链 · Infinity Loop
Git 版本控制	4 个区域 · 6 大命令 · GitHub 工作流
从代码到线上	Cursor 项目级 · Vercel 一键部署

开场案例：CrowdStrike 全球宕机事件（2024）

"史上规模最大的 IT 中断事件之一。" ——科技媒体普遍评价，2024-07-19

CrowdStrike Falcon：全球市占率最高的终端安全软件，部署于数亿台企业设备

时间线	事件
2024-07-19 04:09 UTC	向全球 Falcon 传感器推送内容配置更新（Channel File 291）
04:09 起	全球运行 Windows 的 Falcon 设备触发蓝屏（BSOD），无法启动
数小时内	约 850 万台 Windows 设备同时崩溃
航空	Delta、美联航等航班大规模停飞，累计取消 / 延误超 1 万次
医疗	多国医院手术被迫推迟，急救调度系统受影响
经济损失	标普 500 企业估计损失约 54 亿美元
修复方式	无法远程修复 ，需逐台人工进入安全模式删除文件

根因分析：DevOps 流程的系统性缺失

直接原因：Channel File 291 包含逻辑错误，触发 Windows 内核崩溃——而这个文件被同时推送给了全球所有设备

深层原因：

- **无分阶段发布** (Staged Rollout)：更新同时推向所有设备，没有金丝雀发布、灰度验证
- **测试覆盖不足**：内容配置更新的测试标准低于内核驱动，未覆盖触发场景
- **无自动回滚**：更新一旦推送，发现问题后无法远程撤回
- **修复成本极高**：蓝屏后系统无法远程操作，只能人工逐台处理

今天学什么：

Git (版本追踪 + 可回滚) + CI/CD 流水线 (分阶段测试部署) + Vercel (自动化 + 回滚) ← 正是防止上述灾难的现代工具链

本讲路线图

模块	核心问题	工具	时长
PART 1	什么是 DevOps? 为什么要用?	理念 + 工具链	~10 min
PART 2	如何用 Git 追踪和管理代码?	Git + GitHub	~25 min
PART 3	如何把代码快速部署到线上?	Cursor + Vercel	~15 min

三个部分串联成一条完整的 "写代码 → 追踪版本 → 部署上线" 链路

PART 1

DevOps 与现代开发文化

Dev + Ops + 文化 · 工具 · 实践

DevOps 是什么?

DevOps = 开发 (Dev) + 运维 (Ops) + 文化/工具/实践

打破开发团队与运维团队之间的壁垒, 让软件交付更快、更稳定

传统模式 (Knight Capital 时代):

开发写完代码 → 扔给运维 → 运维手动部署 → 出问题互相推责

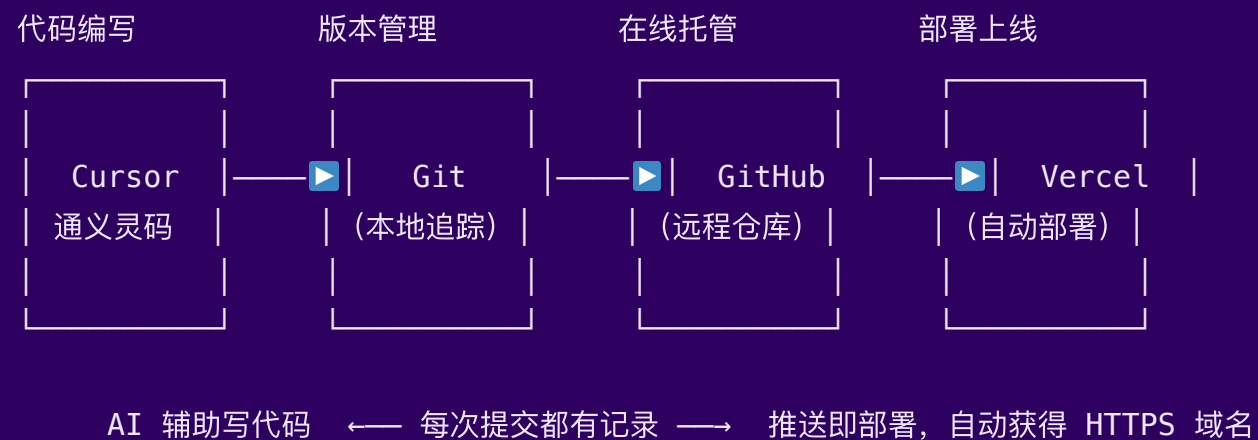
DevOps 模式 (现代):

代码提交 → 自动测试 → 自动构建 → 自动部署 → 监控反馈 → 持续改进

↑ _____ 持续循环 (Infinity Loop) _____ ↑

核心价值: Knight Capital 如果有自动化部署流水线, 旧代码绝不可能悄悄留在一台服务器上

我们的实验工具链



这条链路也是当前最主流的前端/全栈小团队开发工作流

PART 2

| Git 版本控制

4 个区域 · 6 大命令 · GitHub 工作流

Git 的四个区域

工作区 (Working Directory)

↓ `git add`

暂存区 (Index / Staging Area)

← "准备提交"的快照

↓ `git commit`

本地仓库 (Local Repository)

← 完整的版本历史, 存在 `.git` 目录

↓ `git push`

远程仓库 (Remote Repository)

← GitHub / GitLab 上的副本, 团队共享

↑ `git pull` / `git fetch`

类比:

- 工作区 = 你的桌面草稿纸
- 暂存区 = 你决定要装进信封的内容
- 本地仓库 = 你自己的档案柜 (有完整历史)
- 远程仓库 = 办公室公用的云端文件柜

6 个最常用的 Git 命令

1. 克隆远程仓库到本地

```
git clone https://github.com/username/repo.git
```

2. 将文件添加到暂存区 (准备提交)

```
git add filename.py      # 指定文件
```

```
git add .                # 当前目录全部文件
```

3. 提交到本地仓库 (附上有意义的提交信息!)

```
git commit -m "feat: 添加用户登录功能"
```

4. 推送到远程仓库

```
git push origin main
```

5. 从远程仓库拉取最新代码 (= fetch + merge)

```
git pull origin main
```

6. 切换分支 / 查看分支

```
git checkout -b feature/login  # 创建并切换到新分支
```

```
git checkout main              # 切换回主分支
```

查看与撤销：日常必备命令

— 查看状态 —

```
git status      # 查看工作区和暂存区的变动
git log --oneline # 查看简洁的提交历史
git diff        # 查看未暂存的修改内容
```

— 撤销操作 —

```
git checkout -- filename.py # 撤销工作区的修改（危险！不可恢复）
git reset HEAD filename.py  # 将文件从暂存区移出（不删除修改）
git reset --soft HEAD~1     # 撤销上一次 commit（保留修改在暂存区）
```

— 远程操作 —

```
git remote -v      # 查看远程仓库地址
git fetch origin    # 下载远程更新（但不合并）
git merge origin/main # 合并远程分支到当前分支
```

⚠️ `git reset --hard` 会永久丢弃修改，操作前请确认！有 Git 记录才能找回历史

GitHub 团队协作工作流



关键习惯：每次 `commit` 只做一件事，写清楚 commit message——这是 Knight Capital 事故的反面教训

动手实践 (5 分钟)

任务：创建你的第一个 Git 仓库并推送到 GitHub

1. 创建本地项目目录

```
mkdir my-first-repo && cd my-first-repo  
git init
```

2. 创建文件并提交

```
echo "# 我的第一个仓库" > README.md  
git add README.md  
git commit -m "init: 初始化项目"
```

3. 在 GitHub 创建同名仓库 (不要勾选初始化选项)

4. 关联远程仓库并推送

```
git remote add origin https://github.com/你的用户名/my-first-repo.git  
git push -u origin main
```

✅ 成功标志：在 GitHub 页面看到你的 README.md 内容

🗨️ 停下来想一想

"如果你是 Knight Capital 的技术总监，你会要求团队做哪些改变来防止这次事故？"

从 Git 和 DevOps 角度思考：

改进方向	具体措施	对应 Git/DevOps 实践
版本追踪	每个版本都有唯一标识	<code>git tag v1.0.0</code> 打标签
自动化部署	不能靠人工 SSH 逐台操作	CI/CD 流水线
快速回滚	出问题 5 分钟内恢复	<code>git revert</code> / 部署回滚
分支保护	main 分支不能直接推送	Branch Protection Rules

💡 Knight Capital 的教训，正是今天 DevOps 文化兴起的原因之一

PART 3

| 从代码到线上

Cursor 项目级 · Vercel 一键部署

Cursor: 项目级 AI 编程

上一讲我们学了 Cursor 的基础快捷键，这里重点看**项目级**功能：

Ctrl+Enter：扫描整个项目

普通 AI 问答 (Ctrl+L):

"这个函数怎么优化?" → AI 只看当前文件的上下文

项目级 AI (Ctrl+Enter):

"这个项目的数据库连接逻辑在哪里?"

→ AI 扫描所有文件，找到 application.yml + DataSource 配置

→ 给出跨文件的分析答案

实践场景：拿到一个陌生开源项目

1. 用 `git clone` 下载到本地，用 Cursor 打开
2. `Ctrl+Enter`：问"这个项目的整体架构是什么？入口在哪里？"
3. AI 帮你生成项目导览，比自己读代码快 10 倍

Vercel: 从 GitHub 到线上的 3 步

Vercel 是专为前端和全栈项目设计的零配置部署平台

步骤 1: 连接 GitHub

登录 vercel.com → 点击 "New Project" → 授权 GitHub

步骤 2: 导入仓库

选择你的仓库 → 选择框架 (Next.js / Vue / React / 静态网站)

→ Vercel 自动检测构建命令

步骤 3: 部署完成

点击 Deploy → 约 1-2 分钟 → 自动获得 <https://your-project.vercel.app>

特性	说明
自动触发	每次 <code>git push</code> 到 main, 自动重新部署
预览部署	每个 Pull Request 自动生成独立预览链接
自定义域名	免费支持绑定自己的域名 (加 CNAME 记录)
免费额度	个人项目完全免费

完整 workflow 回顾：从本地到线上

1. 本地用 Cursor 开发功能
↓
2. `git add . && git commit -m "feat: xxx"`
↓
3. `git push origin main`
↓
4. Vercel 自动检测到新提交 → 触发 CI/CD
↓
5. 构建完成 → 新版本自动上线 (约 1-2 分钟)
↓
6. 访问 `https://your-project.vercel.app` 验证

对比 Knight Capital: 手动 SSH 部署 8 台服务器 → 漏 1 台 → \$440M 损失
现代工作流: 一次 `git push`, Vercel 自动更新所有实例

动手实践：部署你的第一个网站（8 分钟）

任务：将一个简单的网页部署到 Vercel

```
<!-- 创建 index.html -->
<!DOCTYPE html>
<html>
<head><title>我的第一个网站</title></head>
<body>
  <h1>Hello, Vercel!</h1>
  <p>部署时间：2026 年春季学期</p>
</body>
</html>
```

1. 在本地创建 `index.html`，`git init && git add . && git commit && git push`
2. 登录 `vercel.com`，New Project → 导入上一步的仓库
3. 点击 Deploy，等待约 1 分钟
4. **挑战**：修改 `index.html` 内容，`git push` 后观察 Vercel 自动重新部署

✅ 成功标志：通过 Vercel 提供的 HTTPS 链接访问到你的网页

课堂总结：工具链如何防止 Knight Capital 式灾难

Knight Capital 的问题	现代工具链的解决方案
旧代码无人知晓存在哪里	<code>git log</code> 完整追踪每个文件的每次变更
手动逐台部署，容易漏掉	Vercel 自动化部署，一次 push 全部更新
找不到上一个稳定版本	<code>git tag v1.0</code> 标记稳定版，随时 <code>git checkout</code>
无监控，异常持续 45 分钟	Vercel 部署日志 + 错误告警（实时）

Git 的核心价值：让软件历史成为可查询、可回溯、可协作的数据库而不是 Knight Capital 那种"不知道哪台服务器跑着什么代码"的黑箱

作业（二选一）

选题 A：Git 协作 workflow 实践

1. 在 GitHub 上创建一个仓库，邀请同学作为 Collaborator
2. 各自创建 `feature/xxx` 分支，开发不同功能，通过 Pull Request 合并
3. 故意制造一次合并冲突，记录解决过程
4. 提交：仓库链接 + 冲突解决过程截图 + 300 字总结

选题 B：自动化部署流水线

1. 创建一个简单的网站项目（静态 HTML 或 Vue/React 均可）
2. 连接 GitHub + Vercel，实现 `git push` 触发自动部署
3. 完整演示：修改代码 → push → Vercel 自动上线的过程
4. 提交：Vercel 部署链接 + 操作过程截图 + 反思（哪个步骤最容易出错）

三分钟回顾

问题	答案
Knight Capital 亏损的直接原因?	1 台服务器未更新, 运行旧代码, 无版本控制
Git 的四个区域?	工作区 → 暂存区 → 本地仓库 → 远程仓库
提交时最重要的习惯是?	每次只做一件事, 写清楚 commit message
<code>git reset --hard</code> 的风险?	永久丢弃未提交修改, 不可恢复
Vercel 自动部署的触发条件?	<code>git push</code> 到关联的 GitHub 仓库
DevOps 的核心价值?	打破开发/运维壁垒, 持续交付, 快速回滚

下节预告

第 8-03 讲：AI 编程进阶与 Agent 时代

2024 年，有人宣称 AI 能独立完成工程师的工作——这是真的吗？
人与 AI 的关系，究竟会走向哪里？

南京大学 · 《软件工程与计算 II》 · 2026 春季学期

第 8-02 讲 开发工具链与 Git 工作流

本讲核心收获

- └ DevOps: 自动化 + 持续交付, 彻底告别手动部署的黑箱
- └ Git: 4 个区域 + 6 大命令, 让代码历史可查可回溯
- └ Vercel: `git push` 即部署, 现代工作流的终点

版本控制是工程师最重要的职业习惯之一